

WIPRO SELENIUM

MODULE 3 INDIVIDUAL LAB WORKBOOK

Python BDD RESTful Automations

Name :Debasmita Majhi

Module: Module->3 Python BDD Restful Automations

Module 3 – Individual Lab Work Record

This workbook follows the required conventional laboratory-workbook format. The source code below is attached exactly as it appears in the uploaded Module 3 ZIP file. No source code has been rewritten.

Experiments

3.1 – REST API – Requests and GET

3.2 – Status Code, Headers and JSON

3.3 – POST Request with Payload and Headers

3.4 – PUT, PATCH and DELETE

3.5 – Sessions, Cookies and Authentication

3.6 – Configuration, Reusable Data and MySQL Concept

3.7 – BDD with Behave

3.8 – Hooks, Tags and Allure Reporting

Experiment 3.1 – REST API – Requests and GET

1. Problem Statement

Send a GET request to a REST API and inspect the returned response.

2. Objective

To understand basic REST API automation using Python Requests and validate a successful GET response.

3. Tools, Software and Concepts Used

Tools / Software: Python, Requests, REST API, JSONPlaceholder

Concepts: REST API, HTTP GET, status code, JSON response

4. Source Code / Implementation Steps

experiment_3_1.py

```
import requests

# GET request
response = requests.get("https://jsonplaceholder.typicode.com/posts/1")

# Print response status
print("Status Code:", response.status_code)

# Print response data as JSON
print("Response JSON:")
print(response.json())
```

Execution command:

```
python experiment_3_1.py
```

5. Output with Clearly Labelled Screenshot

Screenshot – Experiment 3.1: REST API – Requests and GET

[INSERT ACTUAL OUTPUT SCREENSHOT HERE]

Screenshot – Experiment 3.1: REST API – Requests and GET

Insert the actual screenshot from your execution here.

6. Result

The experiment was implemented using the exact source code from the Module 3 project.

7. Observation

The experiment demonstrated the intended API automation or BDD concept using the project source code.

8. Conclusion

Experiment 3.1 demonstrated REST API – Requests and GET using the exact project implementation.

Experiment 3.2 – Status Code, Headers and JSON

1. Problem Statement

Retrieve an API response and display its status code, response headers and JSON body.

2. Objective

To validate important components of an HTTP response and save the execution output.

3. Tools, Software and Concepts Used

Tools / Software: Python, Requests, JSON, text output file

Concepts: HTTP status code, response headers, JSON, validation

4. Source Code / Implementation Steps

experiment_3_2.py

```
import requests

# Send GET request
response = requests.get("https://jsonplaceholder.typicode.com/posts/1")

# Prepare output
output = f"""
Status Code: {response.status_code}

Response Headers:
{response.headers}

Response JSON:
{response.json()}

Status Code Validation: {"PASSED" if response.status_code == 200 else "FAILED"}
"""

# Display output in terminal
print(output)

# Save output to a text file
with open("experiment_3_2_output.txt", "w", encoding="utf-8") as file:
    file.write(output)

print("Output saved to experiment_3_2_output.txt")
```

Execution command:

```
python experiment_3_2.py
```

5. Output with Clearly Labelled Screenshot

Screenshot – Experiment 3.2: Status Code, Headers and JSON

[INSERT ACTUAL OUTPUT SCREENSHOT HERE]

Screenshot – Experiment 3.2: Status Code, Headers and JSON

Insert the actual screenshot from your execution here.

6. Result

The experiment was implemented using the exact source code from the Module 3 project.

7. Observation

The experiment demonstrated the intended API automation or BDD concept using the project source code.

8. Conclusion

Experiment 3.2 demonstrated Status Code, Headers and JSON using the exact project implementation.

Experiment 3.3 – POST Request with Payload and Headers

1. Problem Statement

Send a POST request containing JSON data and validate the created-resource response.

2. Objective

To understand API data submission using a JSON payload and request headers.

3. Tools, Software and Concepts Used

Tools / Software: Python, Requests, JSON, REST API

Concepts: HTTP POST, payload, Content-Type, response validation

4. Source Code / Implementation Steps

experiment_3_3.py

```
import requests

url = "https://jsonplaceholder.typicode.com/posts"

payload = {
    "title": "Python API Test",
    "body": "This is a POST request",
    "userId": 1
}

headers = {
    "Content-Type": "application/json"
}

response = requests.post(
    url,
    json=payload,
    headers=headers
)

output = f"""
Status Code: {response.status_code}

Response Headers:
{response.headers}

Response JSON:
{response.json()}

POST Request Validation: {"PASSED" if response.status_code == 201 else "FAILED"}
"""

print(output)

with open("experiment_3_3_output.txt", "w", encoding="utf-8") as file:
    file.write(output)
```

```
print("Output saved to experiment_3_3_output.txt")
```

Execution command:

```
python experiment_3_3.py
```

5. Output with Clearly Labelled Screenshot

Screenshot – Experiment 3.3: POST Request with Payload and Headers

[INSERT ACTUAL OUTPUT SCREENSHOT HERE]

Screenshot – Experiment 3.3: POST Request with Payload and Headers

Insert the actual screenshot from your execution here.

6. Result

The experiment was implemented using the exact source code from the Module 3 project.

7. Observation

The experiment demonstrated the intended API automation or BDD concept using the project source code.

8. Conclusion

Experiment 3.3 demonstrated POST Request with Payload and Headers using the exact project implementation.

Experiment 3.4 – PUT, PATCH and DELETE

1. Problem Statement

Perform update and delete operations on a REST API resource.

2. Objective

To understand the main HTTP methods used for updating and deleting API resources.

3. Tools, Software and Concepts Used

Tools / Software: Python, Requests, REST API, JSON

Concepts: HTTP PUT, PATCH, DELETE, response validation

4. Source Code / Implementation Steps

experiment_3_4.py

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

# PUT request
put_data = {
    "id": 1,
    "title": "Updated Title",
    "body": "Updated using PUT",
    "userId": 1
}

put_response = requests.put(url, json=put_data)

# PATCH request
patch_data = {
    "title": "Partially Updated Title"
}

patch_response = requests.patch(url, json=patch_data)

# DELETE request
delete_response = requests.delete(url)

output = f"""
PUT REQUEST
Status Code: {put_response.status_code}
Response: {put_response.json()}

PATCH REQUEST
Status Code: {patch_response.status_code}
Response: {patch_response.json()}

DELETE REQUEST
```

```
Status Code: {delete_response.status_code}
```

```
PUT Validation: {"PASSED" if put_response.status_code == 200 else "FAILED"}
```

```
PATCH Validation: {"PASSED" if patch_response.status_code == 200 else "FAILED"}
```

```
DELETE Validation: {"PASSED" if delete_response.status_code == 200 else "FAILED"}
```

```
"""
```

```
print(output)
```

```
with open("experiment_3_4_output.txt", "w", encoding="utf-8") as file:  
    file.write(output)
```

```
print("Output saved to experiment_3_4_output.txt")
```

Execution command:

```
python experiment_3_4.py
```

5. Output with Clearly Labelled Screenshot

Screenshot – Experiment 3.4: PUT, PATCH and DELETE

[INSERT ACTUAL OUTPUT SCREENSHOT HERE]

Screenshot – Experiment 3.4: PUT, PATCH and DELETE

Insert the actual screenshot from your execution here.

6. Result

The experiment was implemented using the exact source code from the Module 3 project.

7. Observation

The experiment demonstrated the intended API automation or BDD concept using the project source code.

8. Conclusion

Experiment 3.4 demonstrated PUT, PATCH and DELETE using the exact project implementation.

Experiment 3.5 – Sessions, Cookies and Authentication

1. Problem Statement

Create a Requests session, receive a cookie and validate that the cookie is stored in the session.

2. Objective

To understand session management and cookies in API automation.

3. Tools, Software and Concepts Used

Tools / Software: Python, Requests Session, HTTP cookies

Concepts: Session, cookies, persistent request state

4. Source Code / Implementation Steps

experiment_3_5.py

```
import requests

session = requests.Session()

# Send GET request using session
response = session.get(
    "https://httpbin.org/cookies/set/student/module3"
)

# Get cookies from session
cookies = session.cookies

# Prepare output
output = f"""
Status Code: {response.status_code}

Session Cookies:
{cookies}

Cookie Value:
{cookies.get("student", "Not Found")}

Session Validation: {"PASSED" if "student" in cookies else "FAILED"}
"""

print(output)

# Save output
with open("experiment_3_5_output.txt", "w", encoding="utf-8") as file:
    file.write(output)

print("Output saved to experiment_3_5_output.txt")
```

Execution command:

```
python experiment_3_5.py
```

5. Output with Clearly Labelled Screenshot

Screenshot – Experiment 3.5: Sessions, Cookies and Authentication

[INSERT ACTUAL OUTPUT SCREENSHOT HERE]

Screenshot – Experiment 3.5: Sessions, Cookies and Authentication

Insert the actual screenshot from your execution here.

6. Result

The experiment was implemented using the exact source code from the Module 3 project.

7. Observation

The experiment demonstrated the intended API automation or BDD concept using the project source code.

8. Conclusion

Experiment 3.5 demonstrated Sessions, Cookies and Authentication using the exact project implementation.

Experiment 3.6 – Configuration, Reusable Data and MySQL Concept

1. Problem Statement

Use configurable API information and reusable test data to validate an API response.

2. Objective

To understand basic configuration/data-driven API automation and the role of database data in testing.

3. Tools, Software and Concepts Used

Tools / Software: Python, Requests, test data, MySQL concept

Concepts: Base URL, endpoint, reusable test data, validation, database concept

4. Source Code / Implementation Steps

experiment_3_6.py

```
import requests

# Global configuration
BASE_URL = "https://jsonplaceholder.typicode.com"
ENDPOINT = "/posts/1"

# Reusable data
test_data = {
    "expected_status": 200,
    "expected_user_id": 1
}

# Send API request
response = requests.get(BASE_URL + ENDPOINT)

data = response.json()

# Validate response
status_check = response.status_code == test_data["expected_status"]
user_check = data["userId"] == test_data["expected_user_id"]

output = f"""
API CONFIGURATION
Base URL: {BASE_URL}
Endpoint: {ENDPOINT}

RESPONSE
Status Code: {response.status_code}
User ID: {data["userId"]}
Title: {data["title"]}

VALIDATION
Status Code Validation: {"PASSED" if status_check else "FAILED"}
User ID Validation: {"PASSED" if user_check else "FAILED"}

MySQL Database:
```

```
Database connection concept demonstrated for API data-driven automation.
"""
```

```
print(output)
```

```
# Save output
with open("experiment_3_6_output.txt", "w", encoding="utf-8") as file:
    file.write(output)
```

```
print("Output saved to experiment_3_6_output.txt")
```

Execution command:

```
python experiment_3_6.py
```

5. Output with Clearly Labelled Screenshot

Screenshot – Experiment 3.6: Configuration, Reusable Data and MySQL Concept

[INSERT ACTUAL OUTPUT SCREENSHOT HERE]

Screenshot – Experiment 3.6: Configuration, Reusable Data and MySQL Concept

Insert the actual screenshot from your execution here.

6. Result

The experiment was implemented using the exact source code from the Module 3 project.

7. Observation

The experiment demonstrated the intended API automation or BDD concept using the project source code.

8. Conclusion

Experiment 3.6 demonstrated Configuration, Reusable Data and MySQL Concept using the exact project implementation.

Experiment 3.7 – BDD with Behave

1. Problem Statement

Create a BDD feature and step definitions to verify a GET API response.

2. Objective

To implement API automation using Behavior-Driven Development with Behave.

3. Tools, Software and Concepts Used

Tools / Software: Python, Requests, Behave, Gherkin

Concepts: Feature, Scenario, Given/When/Then, step definitions

4. Source Code / Implementation Steps

features/api_test.feature

Feature: API Automation

@api

Scenario: Verify GET API response

Given I send a GET request to the posts API

When I receive the API response

Then the response status code should be 200

features/steps/api_steps.py

```
import requests
```

```
from behave import given, when, then
```

```
@given("I send a GET request to the posts API")
```

```
def step_send_request(context):
```

```
    context.response = requests.get(
```

```
        "https://jsonplaceholder.typicode.com/posts/1"
```

```
    )
```

```
@when("I receive the API response")
```

```
def step_receive_response(context):
```

```
    context.status_code = context.response.status_code
```

```
@then("the response status code should be 200")
```

```
def step_verify_status(context):
```

```
    assert context.status_code == 200
```

Execution command:

```
behave
```

5. Output with Clearly Labelled Screenshot

Screenshot – Experiment 3.7: BDD with Behave

[INSERT ACTUAL OUTPUT SCREENSHOT HERE]

Screenshot – Experiment 3.7: BDD with Behave

Insert the actual screenshot from your execution here.

6. Result

The experiment was implemented using the exact source code from the Module 3 project.

7. Observation

The experiment demonstrated the intended API automation or BDD concept using the project source code.

8. Conclusion

Experiment 3.7 demonstrated BDD with Behave using the exact project implementation.

Experiment 3.8 – Hooks, Tags and Allure Reporting

1. Problem Statement

Use Behave hooks and tags and execute the tagged scenario with Allure result generation.

2. Objective

To organize BDD execution using hooks and tags and generate results for reporting.

3. Tools, Software and Concepts Used

Tools / Software: Python, Behave, Allure Behave

Concepts: before_scenario, after_scenario, tags, Allure results

4. Source Code / Implementation Steps

features/environment.py

```
def before_scenario(context, scenario):
    print(f"\nStarting Scenario: {scenario.name}")
```

```
def after_scenario(context, scenario):
    print(f"Finished Scenario: {scenario.name}")
```

features/api_test.feature

Feature: API Automation

```
@api
Scenario: Verify GET API response

    Given I send a GET request to the posts API
    When I receive the API response
    Then the response status code should be 200
```

features/steps/api_steps.py

```
import requests
from behave import given, when, then

@given("I send a GET request to the posts API")
def step_send_request(context):
    context.response = requests.get(
        "https://jsonplaceholder.typicode.com/posts/1"
    )

@when("I receive the API response")
def step_receive_response(context):
    context.status_code = context.response.status_code

@then("the response status code should be 200")
def step_verify_status(context):
    assert context.status_code == 200
```

Execution command:

```
behave --tags=@api
behave -f allure_behave.formatter:AllureFormatter -o allure-results
```

5. Output with Clearly Labelled Screenshot

Screenshot – Experiment 3.8: Hooks, Tags and Allure Reporting

[INSERT ACTUAL OUTPUT SCREENSHOT HERE]

Screenshot – Experiment 3.8: Hooks, Tags and Allure Reporting

Insert the actual screenshot from your execution here.

6. Result

The experiment was implemented using the exact source code from the Module 3 project.

7. Observation

The experiment demonstrated the intended API automation or BDD concept using the project source code.

8. Conclusion

Experiment 3.8 demonstrated Hooks, Tags and Allure Reporting using the exact project implementation.

Overall Conclusion

Module 3 covered REST API automation with Python Requests, HTTP methods, response validation, sessions and cookies, reusable API data, BDD with Behave, hooks, tags and Allure reporting.

Submission Checklist

- ☐ Module name and experiment title included.
- ☐ Problem statement included.
- ☐ Objective included.
- ☐ Tools, software and concepts included.
- ☐ Exact source code attached for every experiment.
- ☐ Implementation/execution steps included.
- ☐ Actual output screenshots inserted and clearly labelled.
- ☐ Result, observation and conclusion completed.